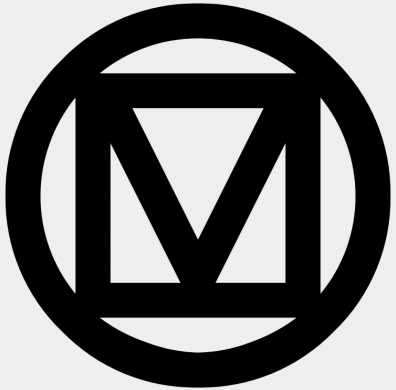
Basics of Android: Project, components

Android Lecture 2

Simona Kurňavová

Today’s Agenda

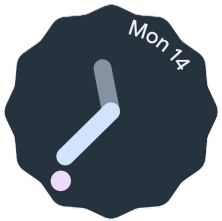
- Android User Interface
- Project Setup & Structure
- Android Basic Components
- UI implementation
- Q&A



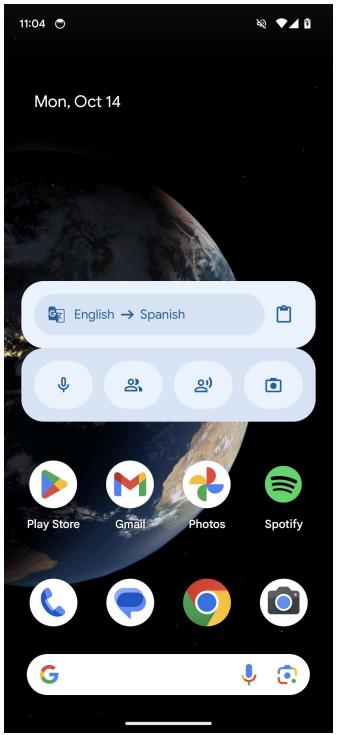
Android User Interface

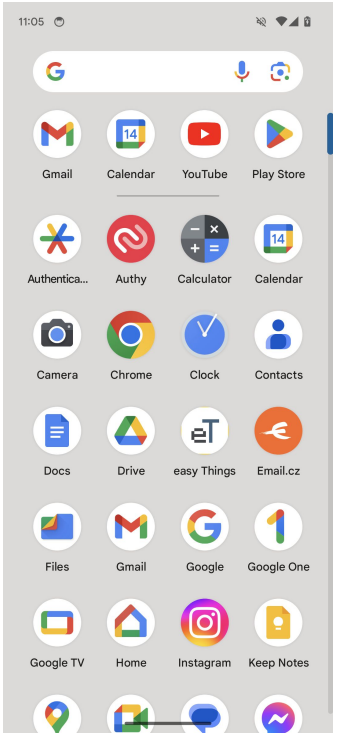
Launcher

- Acts as home screen and app drawer
- Contains apps and widgets
- App widget vs. Widget:
  - Widget is UI item (button, checkbox, etc).
  - App widget is interactive component in launcher/home screen.
- Option of third party launchers (Nova)



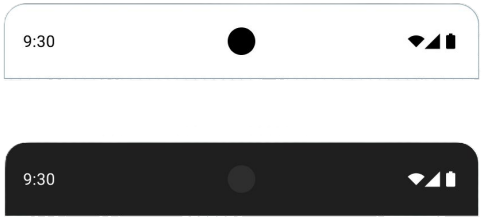
App widgets

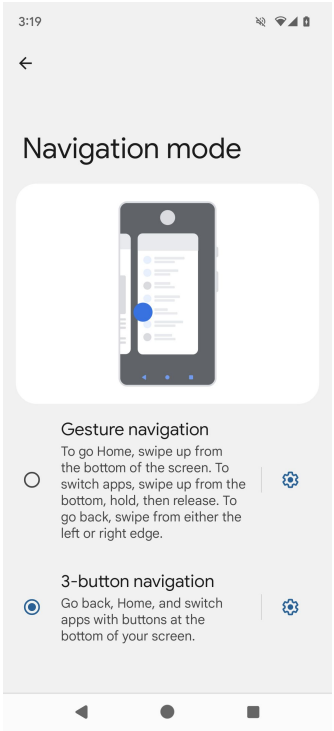


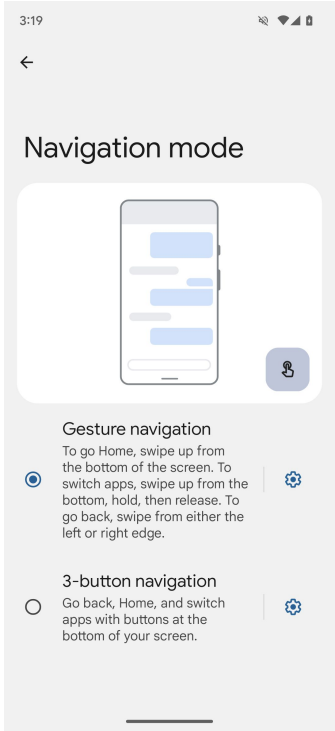


Status bar and navigation

- Purpose of status bar: Displays essential information about the device's current state and notifications.
- Purpose of navigation bar: Provides system-wide controls to navigate through apps and the Android system.

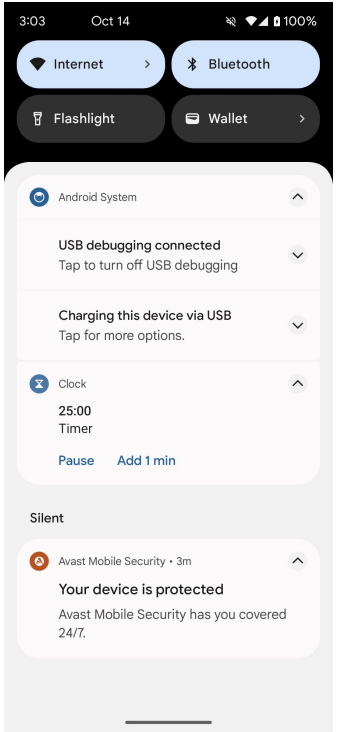


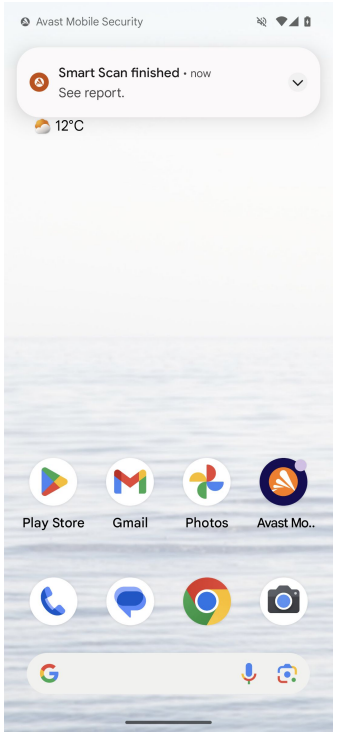




Notifications

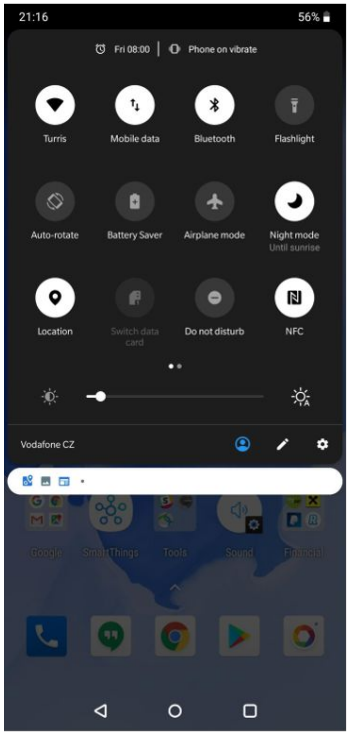
- Alerts that inform users about important events, updates, or actions from apps
- Notification actions (since Android 4.1)
- Can be visible in lock screen (since Android 5.0)
- Notification groups (since Android 7.0)
- Notification channels (since Android 8.0)
- Notification badges (dots) (since Android 8.0)
- *Importance*:
  - Urgent: makes a sound and appears as a heads-up notification.
  - High: makes a sound.
  - Medium: makes no sound.
  - Low: makes no sound and doesn't appear in the status bar.

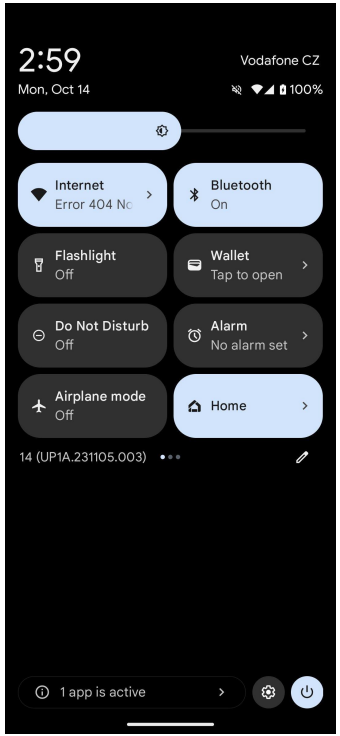


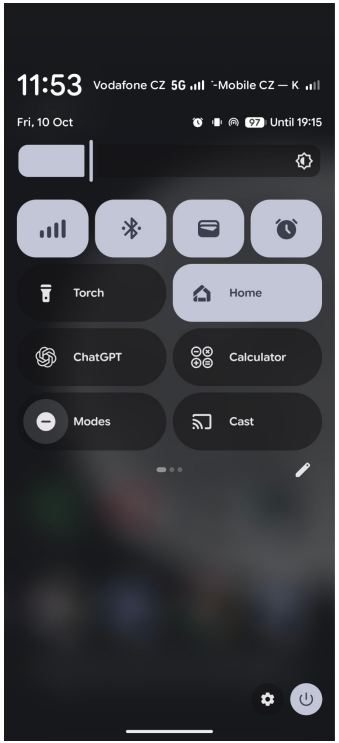


Quick settings

- Since API 16
- Since API 24: Custom tiles
- Purpose of tile action:
  - Used often
  - Fast access needed
  - Ideally both

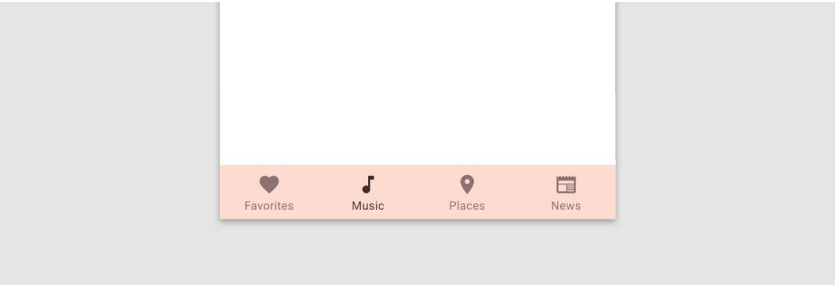


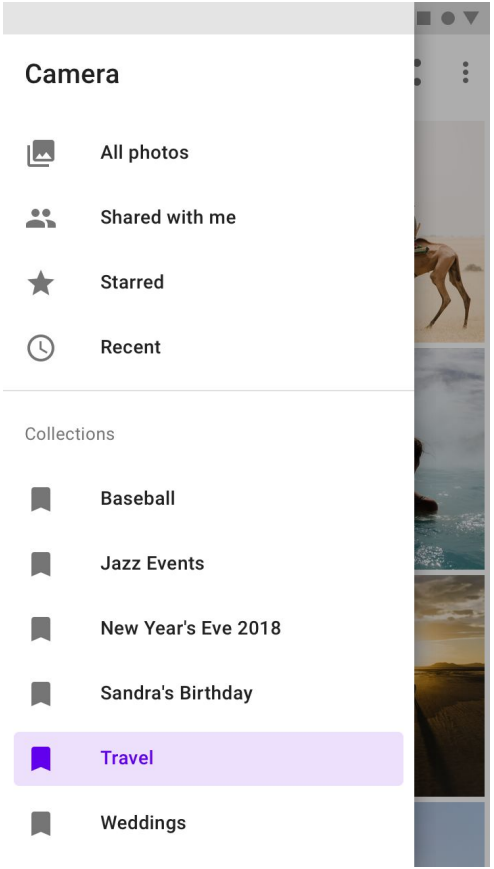




Navigation inside of the app

- Bottom navigation
- Drawer

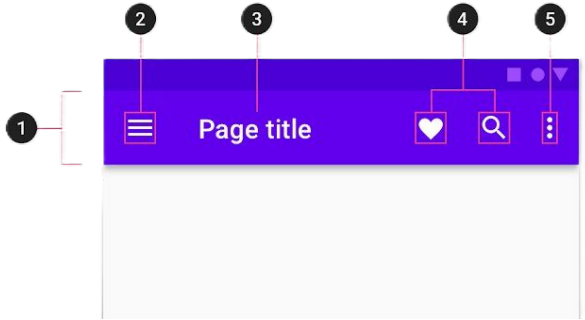




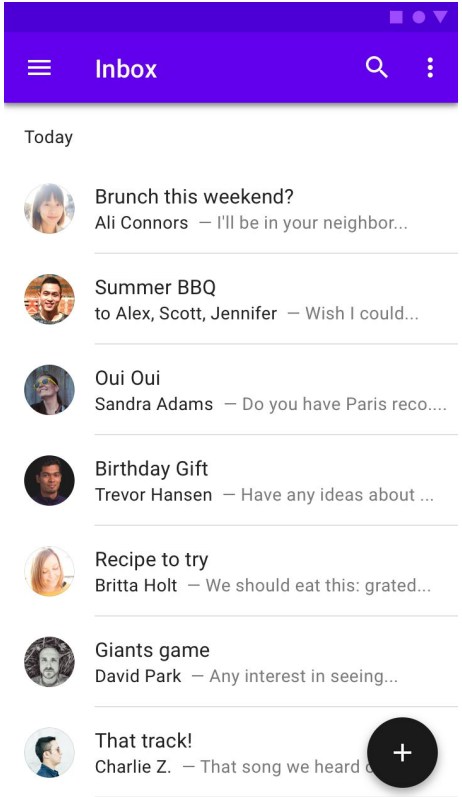
Source1, Source2

Navigation inside of the app

- Toolbar (or app bar)
- Floating action button



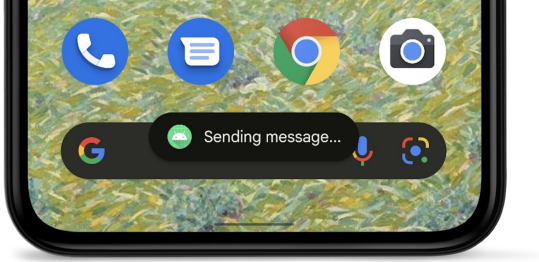
1. Container
2. Navigation icon
3. Title
4. Action items
5. Overflow menu

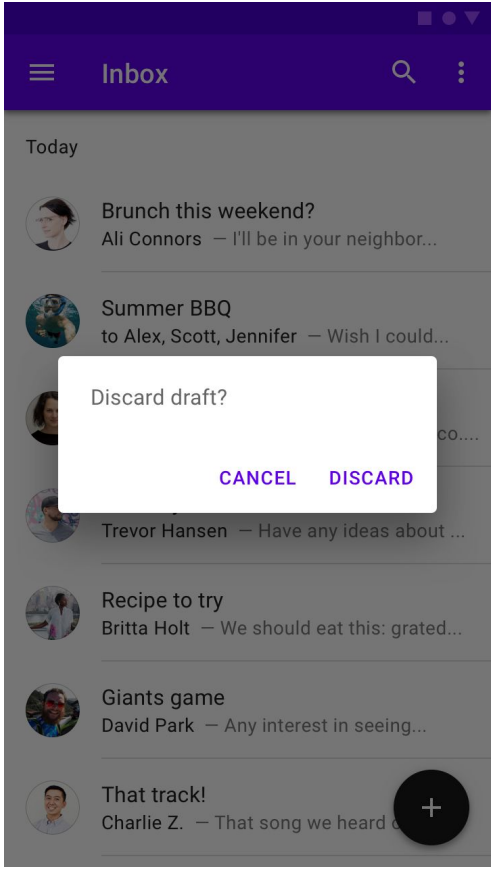


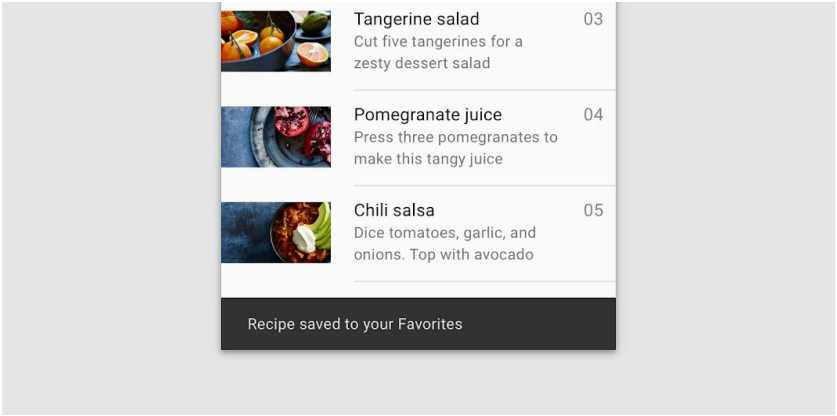
Source1, Source2

In-app messaging

- Dialogs
- Toasts
- Snackbars





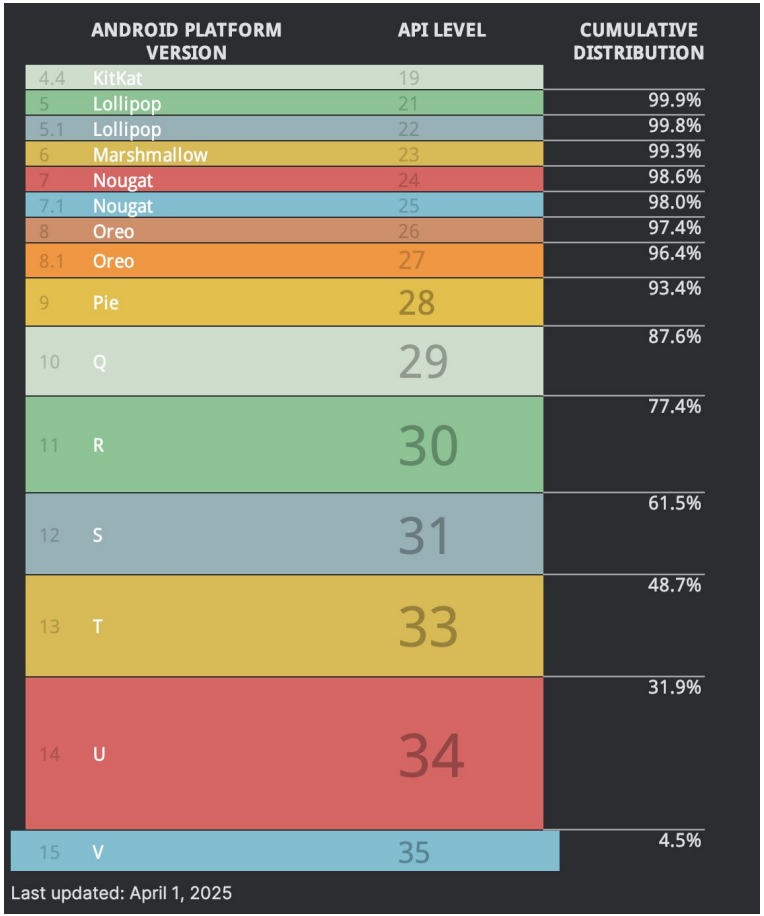


Source1, Source2

Android Project Setup

Consider

- Target audience
- Compatibility
- Native vs. KMP



	ANDROID PLATFORM VERSION	API LEVEL	CUMULATIVE DISTRIBUTION
4.4	KitKat	19	
5	Lollipop	21	99.9%
5.1	Lollipop	22	99.8%
6	Marshmallow	23	99.3%
7	Nougat	24	98.6%
7.1	Nougat	25	98.0%
8	Oreo	26	97.4%
8.1	Oreo	27	96.4%
9	Pie	28	93.4%
10	Q	29	87.6%
11	R	30	77.4%
12	S	31	61.5%
13	T	33	48.7%
14	U	34	31.9%
15	V	35	4.5%

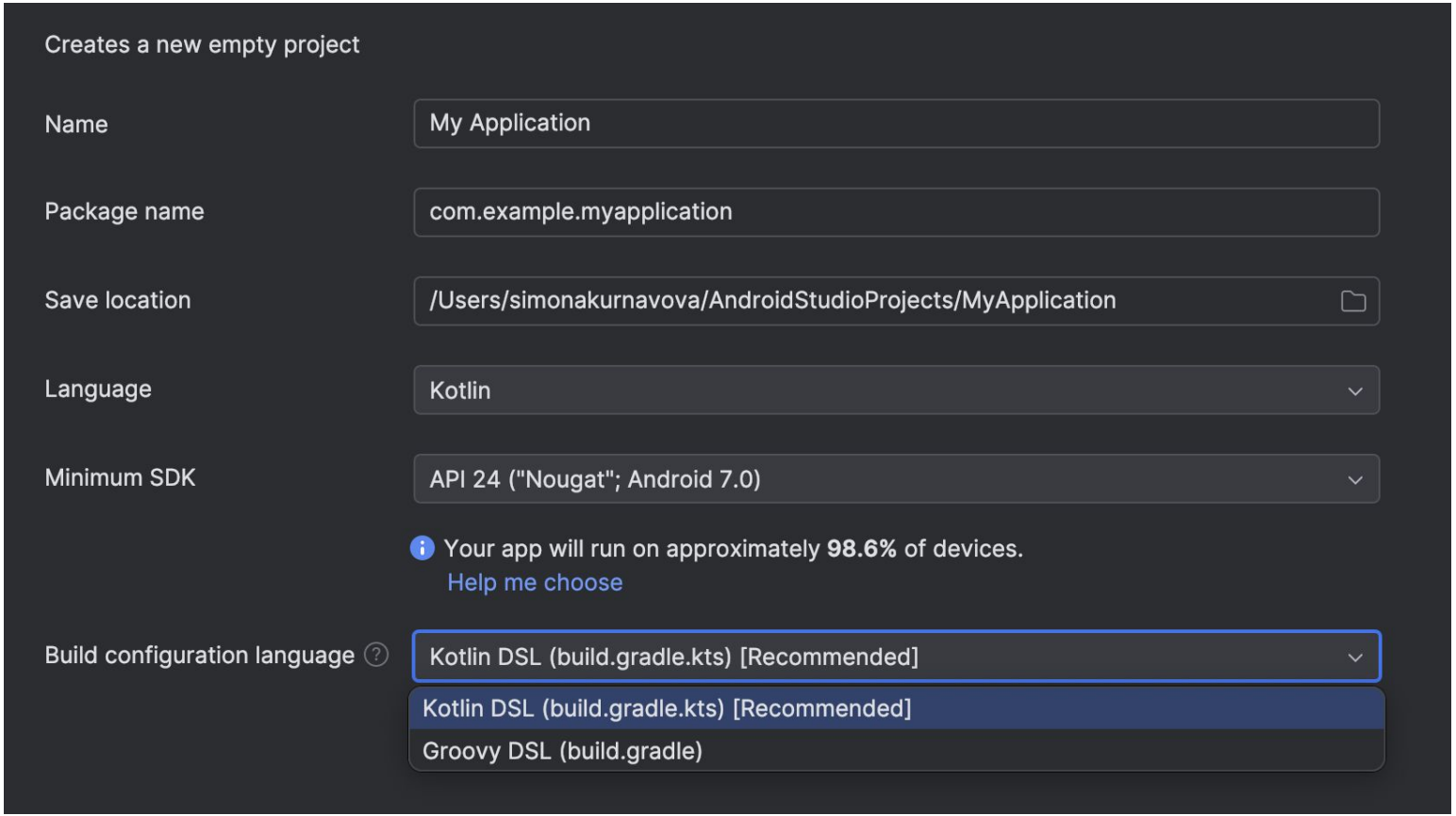
Last updated: April 1, 2025

Source: Android Studio

minSdk, compileSDK, targetSDK

- **minSdk**: Lowest supported SDK
  - Installation on older devices is not possible
  - New features are not available on older APIs
  - Supporting old SDK can take a lot of resources to maintain
  - compatibility API levels checks
  - Testing
- **compileSDK**: Always compile with the latest SDK !
  - Select newest available API at compile time
  - Deprecations
  - Lint checks
- **targetSDK**: Way how system provide forward compatibility
  - Change behavior of the app
  - Runtime permissions handling
  - Menu button deprecation handling

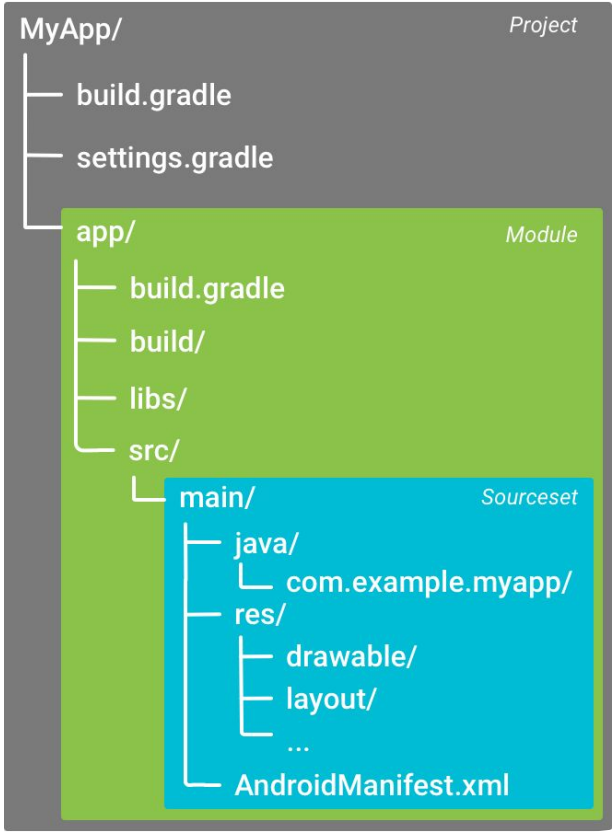
New Project (Android Studio)



Project Structure

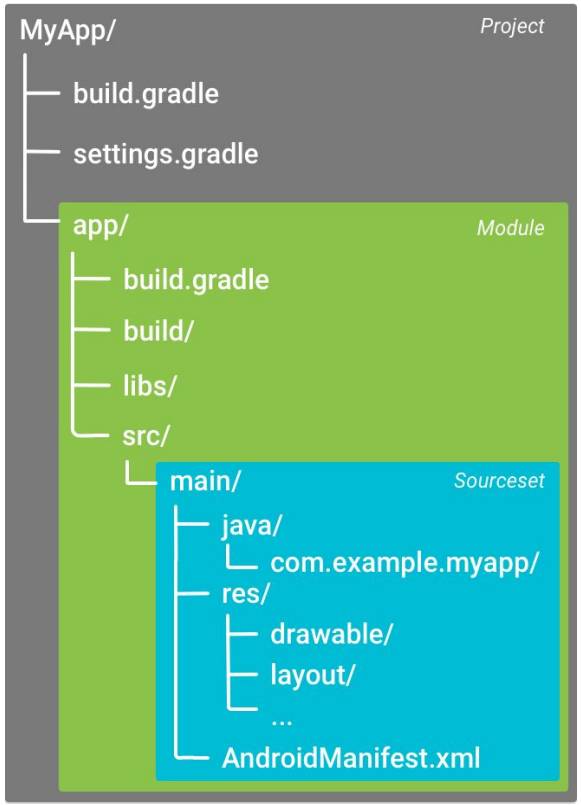
Project structure: Project

- Common configuration for modules
  - Common dependencies versions
  - 3rd party plugins configuration



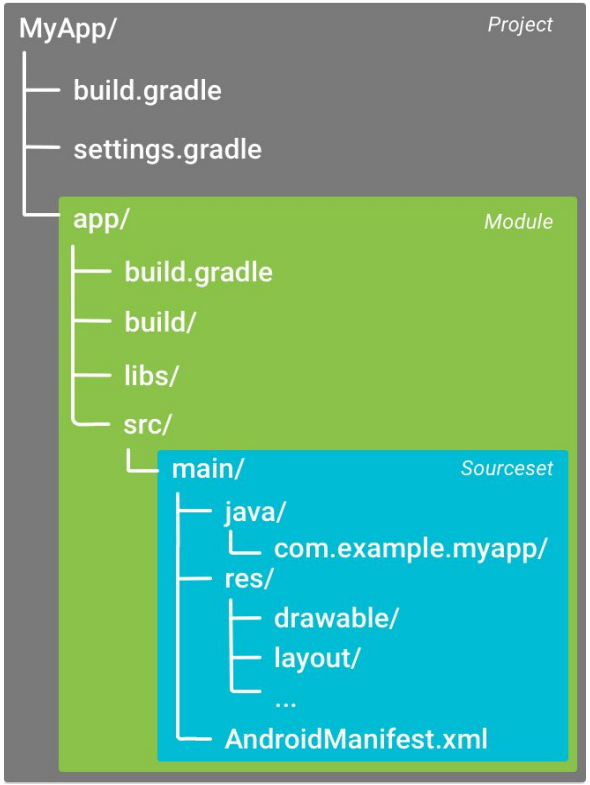
Project structure: Module

- Represents application or library
- Default module “app”
- Small apps can have one module, bigger can be separated into multiple modules by:
  - features
  - layers (UI, domain, data, etc.)
  - or any other logical unit
- Modules can depend on each other (but there should be no circular dependencies)
- Multiple source sets (optional)
  - Different version of same app (paid vs. free, release vs. debug, etc.)



Project structure: Sourceset

- Default “main”
- Source code and resources
- Source code from main source set available everywhere
- Resources can be overridden in different source set



Project level files: **build.gradle**

- Configuration that applies to all modules
- Shared build logic
- Can specify plugin versions globally
- Can specify global tasks

build.gradle (project)

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
plugins {
    id("com.android.application") version "8.7.1" apply false
    id("com.android.library") version "8.7.1" apply false
    id("org.jetbrains.kotlin.android") version "2.0.21" apply false
}
```

Project level files: **settings.gradle**

- List of modules to build
- Specifies where Gradle should find and download plugins
- Can enable Gradle features
- …

settings.gradle

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "MyApp"
include(":app")
```

Project level files: **gradle.properties**

- Project wide gradle fields
- Customization of how it will run
  - Heap size
  - Daemon or not
  - Java_home and java arguments
  - Parallel run
  - Proxy
  - And much more

gradle.properties

```
# Heap size for the Gradle Daemon
org.gradle.jvmargs=-Xmx2048m -XX:MaxPermSize=512m -
XX:+HeapDumpOnOutOfMemoryError -Dfile.encoding=UTF-8

# Enable or disable the Gradle Daemon
org.gradle.daemon=true

# Set the Java home directory (replace with your JDK path)
org.gradle.java.home=/path/to/your/jdk
java.home=/path/to/your/jdk

# Enable parallel execution of tasks
org.gradle.parallel=true

# Enable the use of AndroidX
android.useAndroidX=true
android.enableJetifier=tru

# Enable stack traces for troubleshooting
org.gradle.debug=true
```

Project level files: **local.properties**

- Contains paths to SDK and NDK
- Can’t be shared between developers
- Generated during build, do not modify it manually
- Do not include this file in submitted projects
- Belongs to .gitignore

local.properties

```
## This file must *NOT* be checked into Version Control Systems,
# as it contains information specific to your local configuration.
#
# Location of the SDK. This is only used by Gradle.
# For customization when using a Version Control System, please read the
# header note.
#Tue Jul 01 16:51:14 CEST 2025
sdk.dir=/path/to/sdk
```

Module level files: **build.gradle**

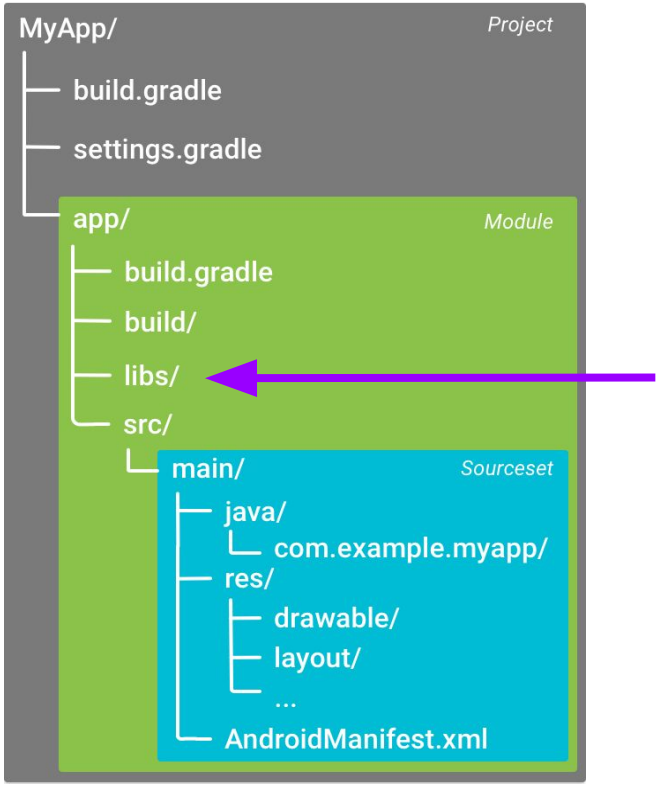
- Configure build setting for specific module
- Defines build variants and their source sets
- applicationId
- Min and target SDK version
- compileSdkVersion and buildToolsVersion
- Dependencies
- Documentation

build.gradle (app)

```gradle
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.example.myapp"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.myapp"
        minSdk = 21
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
    }
}

dependencies {
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")

    // Testing libraries
    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
}
```

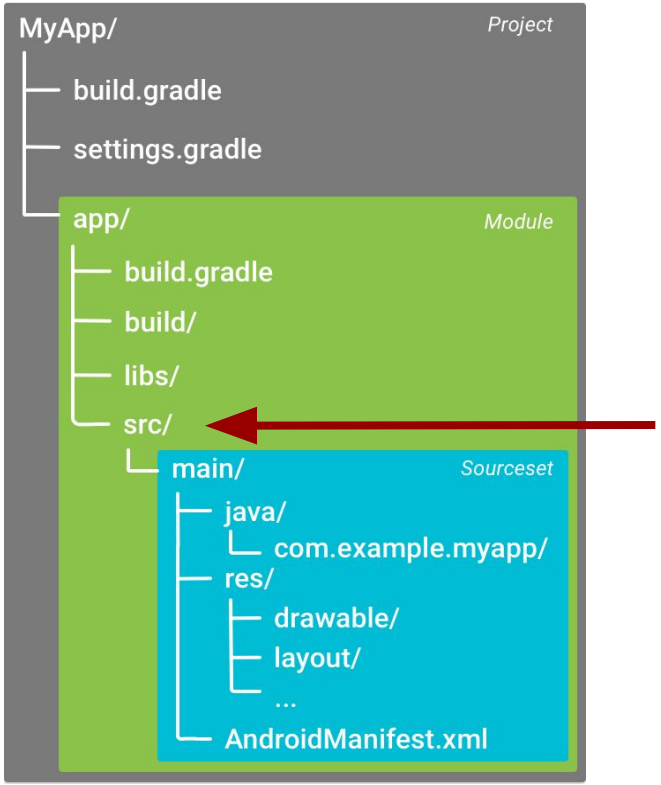
Module level files: libs/

- *.jar libraries
- If it is possible use library as gradle dependency



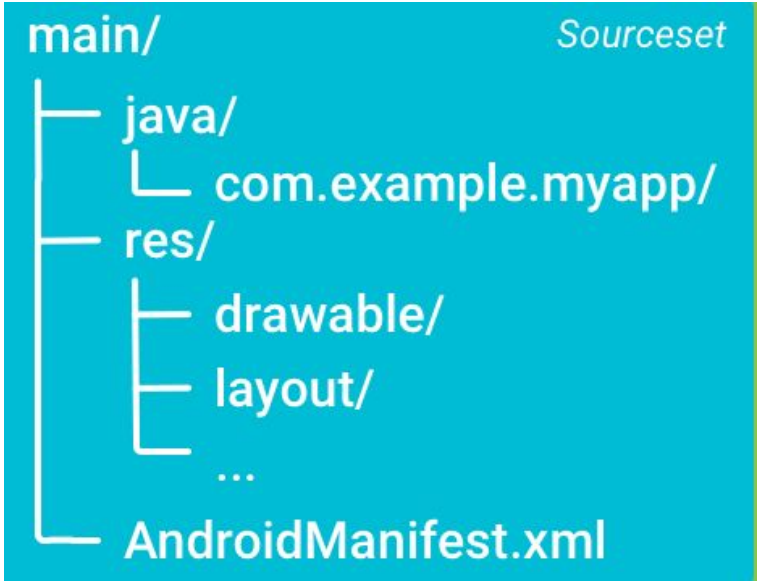
Module level files: **src/**

- Source code
- Resources
- Assets
- Main - default sourceset for all build variants
- Recommended to split code into packages



Source set

- **java/**
  - Source codes
- **res/**
  - Resources
  - Drawables
  - Layouts
  - Values
  - …
- **assets/**
- **AndroidManifest.xml**



Resources

- Strings
- Menu
- Animations
- Icons
- Dimensions
- Drawables
- Mipmap
- Layout (XML)

Resource qualifiers

- Resources in different variants
- Drawable, drawable-mdpi…
- Values, values-cs, values-de
- Layout, layout-sw600dp

Resources: **drawables**

- Bitmaps
- 9-patch png
- State lists
- Vector drawables
  - Since API 21
  - Backward compatibility with support library
- Always prefer vectors over bitmaps

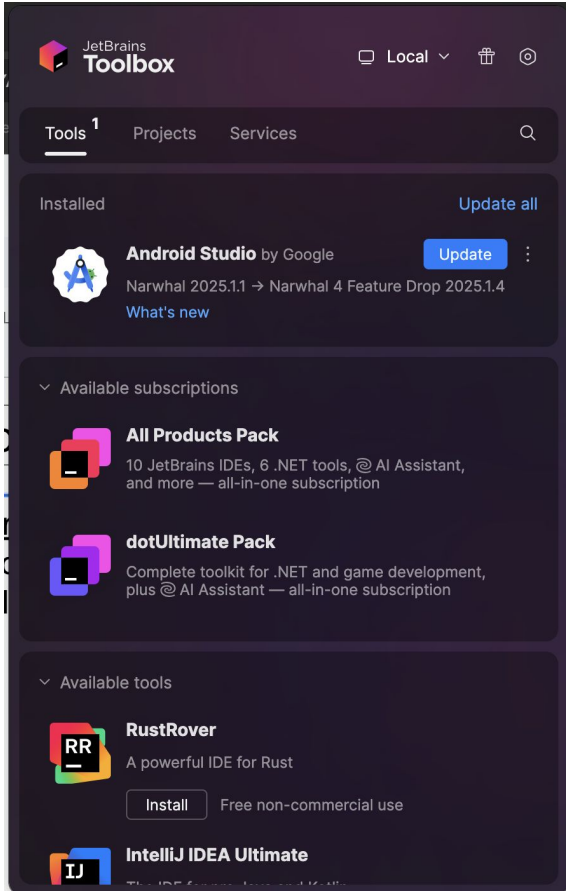
Resources: **Units**

- Dp - density independent pixel
  - On 160dpi screen 1dp = 1px
- Sp - scale independent pixel (fonts)
  - Similar to dp, but scaled by the user’s font size preference
- **Never use px**

Tips & Tricks for Android Studio

Android Studio: General

- **Version**: When in doubt, choose latest stable version
- **JetBrains Toolbox**: https://www.jetbrains.com/toolbox-app/
  - Management tool for JetBrains IDEs
  - Version overview, install/uninstall/update



Android Studio/InteliJ: Shortcuts

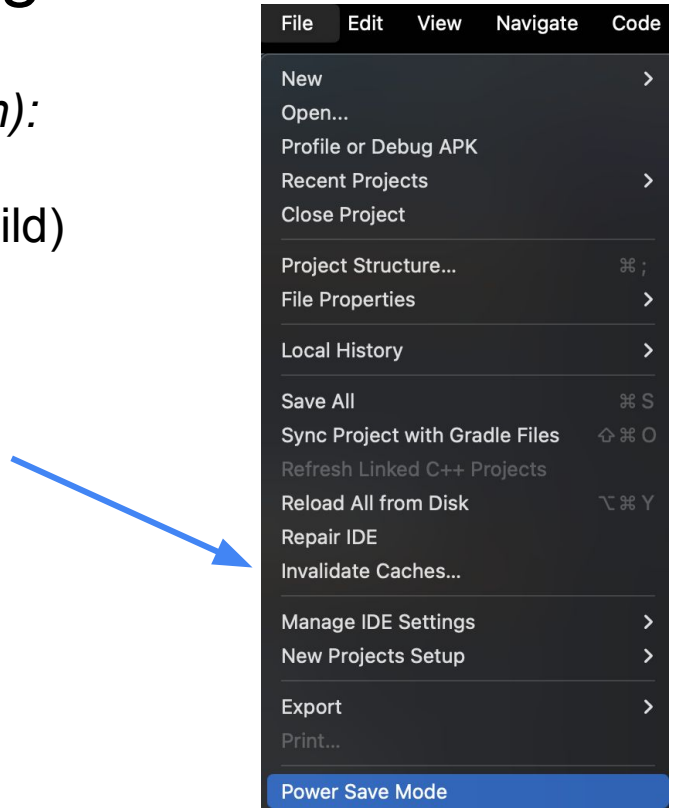
CTRL + SHIFT + F	Find in Files (text search)
2x SHIFT	Search filenames, methods, class names, etc.
CTRL + SHIFT + R	Replace text
CTRL + SHIFT + E	Opens window with previous location that were edited
CTRL + SHIFT + Backspace	Takes you to last edited location
Alt + Enter	Show context actions (action specific to that place)
Alt + Shift	While holding it, you can create multiple cursors and then edit lines all at once *Usage: Add prefix/suffix to multiple lines, deletion of something, etc.*

Note: Substitute **CTRL -> CMD** and **ALT -> Option** on MacOS

Android Studio - Troubleshooting

Build or sync issues (failing for no obvious reason):

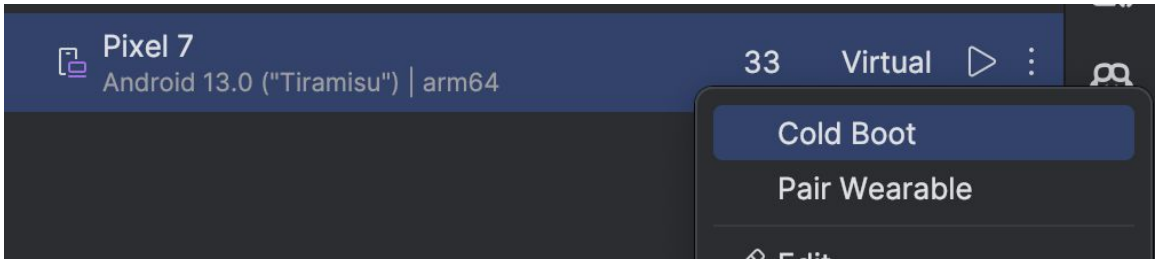
1. Simple restart of Android Studio
2. Clean build and build again (Build - Clean build)
3. File -> Invalidate Caches and restart
4. Manual deletion of Gradle cache - as a last resort; the sync and build after this will be significantly longer
   - Delete contents of ~/.gradle/caches/



Android Studio - Troubleshooting

Emulator not working:

1. Restart of emulator / Android studio
2. Close emulator + cold boot
3. Restart of ADB: `adb kill-server && adb start-server`
4. Force fill ADB: `killall -9 adb`



Logging and debugging

Logs

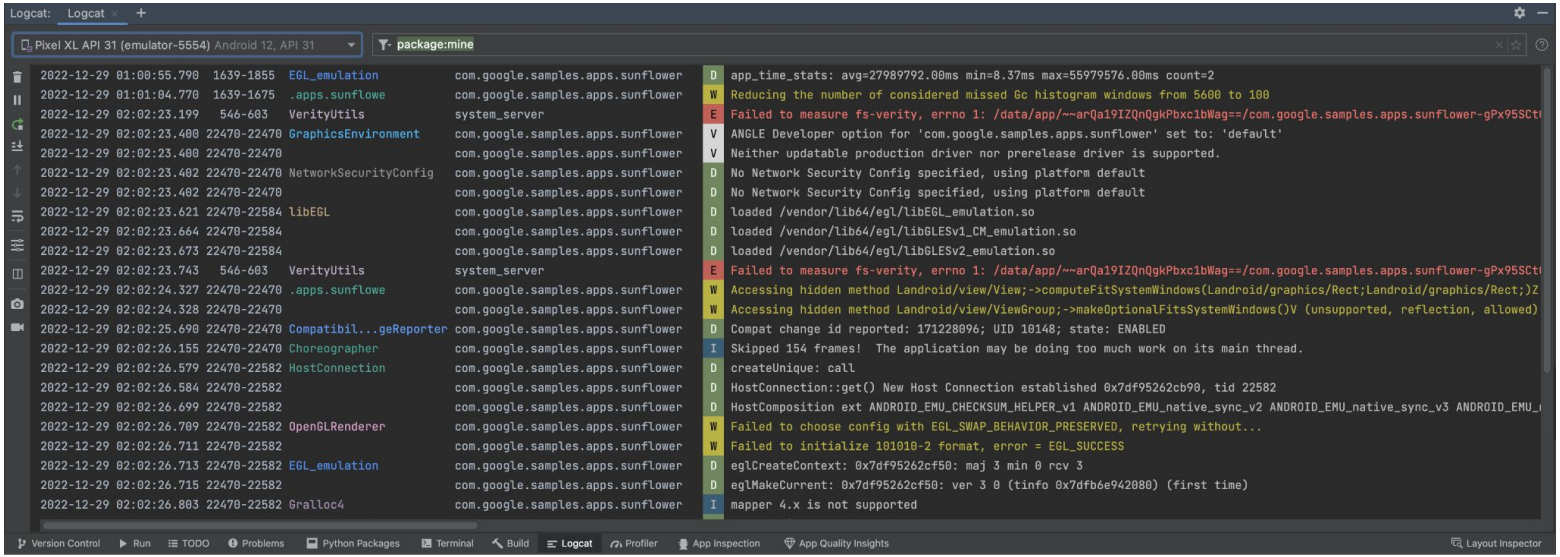
- Documentation
- Logging for application
- Different logging levels: `Log.v()`, `Log.d()`, `Log.i()`, `Log.w()`, `Log.e()`
- **Do not log sensitive data to higher levels!**

Logging

```
private const val TAG = "Feature X"
...

Log.d(TAG, "Hello Logcat")
```

Logs: View

- Logs can be viewed in Android Studio Logcat or in terminal using ADB.
- https://developer.android.com/studio/debug/logcat
- https://developer.android.com/tools/logcat



Crash vs. ANR

Crash = The app stops unexpectedly because of an unhandled exception

- App stops right away and closes
- Shows in Logcat as Exception with stacktrace

ANR (Application Not Responding) = Long-running operations on the main thread

- network, DB query, heavy computation
- app might appear frozen and unresponsive, then dialog “App is not responding” is shown
- Shows in Logcat as “ANR”; often not obvious where it came from

Strict Mode

- Documentation
- Security mechanism/tool
- Raises alarm when database/Network communication is accessed on Main thread
- Can help prevent ANRs
- Can be enabled in Application#onCreate

Strict Mode

```
if (DEVELOPER_MODE) {
    StrictMode.setThreadPolicy(StrictMode.ThreadPolicy.Builder()
            .detectDiskReads()
            .detectDiskWrites()
            .detectNetwork()   // or .detectAll() for all detectable problems
            .penaltyLog()
            .build())
    StrictMode.setVmPolicy(new StrictMode.VmPolicy.Builder()
            .detectLeakedSqlLiteObjects()
            .detectLeakedClosableObjects()
            .penaltyLog()
            .penaltyDeath()
            .build())
}
```

Building blocks of Android application

Android App Components

- Activities
  - It represents a single screen with a user interface
- Services
  - A component that runs in the background to perform long-running operations or to perform work for remote processes; has no UI
- Broadcast receivers
  - A component that lets the system deliver events to the app outside of a regular user flow so the app can respond to system-wide broadcast announcements
- Content providers
  - manages a shared set of app data that you can store in the file system, in a SQLite database, on the web, or on any other persistent storage location that your app can access

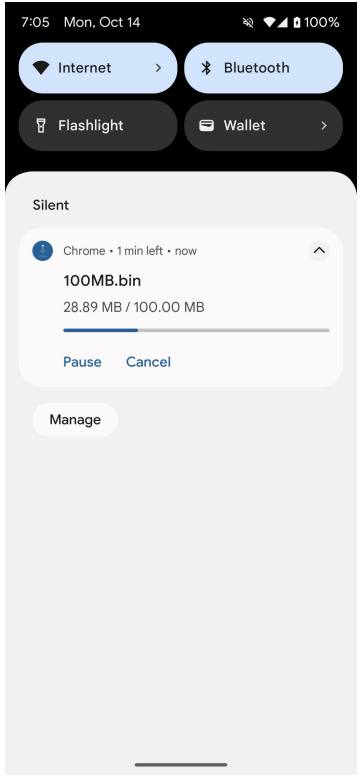
Source

Additional Android Components

- Fragments (docs)
  - Represents a reusable portion of your app's UI
  - **Not needed when using Jetpack Compose**
- Composables (docs)
  - Jetpack Compose UI element
- ViewModel (docs)
  - It exposes state to the UI (Activity, Fragment, Composable) and encapsulates related business logic
- AndroidManifest (docs)
  - Describes essential information about your app to the Android build tools, the Android operating system, and Google Play.
- Intent (docs)
  - Messaging object between components
  -

Service

- No UI
- Optional notification (mandatory for foreground services)
- Operations that are not tight to activity lifecycle
- Long running tasks: Music playback, Download service



Broadcast receiver

- Listens for actions invoked by system or other application
- Static or dynamic registration
- System-wide
- *Examples*:
  - Incoming SMS
  - Low battery, battery percentage changed
  - Connectivity change
  - Headphones connected/disconnected
  - ...

Content provider

- Manage and share application data
- Doesn't specify storage implementation (db, file, web)
- Query or modify data
- Optional permissions
  - Custom permissions who can access data
- Used by system for
  - SMS
  - Contacts
  - Call log
- Initialized before Application
  - Used by AndroidX App Startup library

AndroidManifest.xml

- Essential information about app for OS
- Package name (application unique id)
- Describes components
- Permissions
- Min required API level
- Target SDK
- Supported/required screens, features
- Used for filtering in app store

AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">

    <!-- Specify minimum and target SDK versions -->
    <uses-sdk
        android:minSdkVersion="26"
        android:targetSdkVersion="34" />

    <!-- Declare supported screen sizes and densities -->
    <supports-screens
        android:smallScreens="true"
        android:normalScreens="true"
        android:largeScreens="true"
        android:xlargeScreens="true"
        android:anyDensity="true" />

    <!-- Required hardware features (e.g., camera) -->
    <uses-feature
        android:name="android.hardware.camera"
        android:required="true" />

    <!-- Permission to access the internet -->
    <uses-permission android:name="android.permission.INTERNET" />

    <!-- Application declaration -->
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApp">

        <!-- Main activity declaration -->
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Activity

- *Screen with UI*
- *User interface layer of application*
- Only UI component
- Every activity must be defined in manifest
- Runs on UI (Main) thread
- Lifecycle
- Activity back stack

Source

Activity

- Contains: Views, Fragments, Composables
- *Single activity applications are recommended by Google*

Simple Activity

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Source

Intent

- Asynchronous message between component
- Starts activities
- Starts or binds services
- Sends broadcast

Starting activity with Intent

- Intent describes which activity to start
- Can contain data for new activity
- Flags - manipulation with activity stack

Start an activity

```
val intent = Intent(context, SecondActivity::class.java)
intent.putExtra("key", "value")
intent.putExtra("keyInt", 5)

startActivity(intent)
```

Explicit vs. implicit intent

- ***Explicit intent***
  - Specify component by fully qualified class name
  - Typically component in our application
- ***Implicit intent***
  - Just declare general action to perform
  - Enables multiple apps to handle that action
  - Examples
    - Send email - ACTION_SEND
    - Open browser - ACTION_VIEW
  - If multiple apps are capable to handle intent, system shows picker
  - Intent filters defined in manifest

Implicit Intent - Example

Open URL in browser intent

```
// 1. Parse the URL string into a Uri object
val webpage: Uri = Uri.parse("https://developer.android.com/")

// 2. Create an Intent with ACTION_VIEW
val intent = Intent(Intent.ACTION_VIEW, webpage)

// 3. Check if there is an application that can handle this Intent.
// This is good practice to prevent the app from crashing.
if (intent.resolveActivity(packageManager) != null) {
    // 4. Start the Activity
    startActivity(intent)
} else {
    // Handle error: messaging to user, alternative intent, redirect, etc
}
```

- Project structure
- Hello World application

Context

Context

- android.content.Context
- Abstract class implemented by components
- When using it for some operation, it “tells” from which component the code is being called
- Not all operations are permitted/allowed in all contexts
- **Usage**:
  - Resources access - strings, drawables, …
  - Register/unregister BroadcastReceivers
  - Run Activity
  - Run and binds Services
  - Obtain instances of system services (NotificationManager,

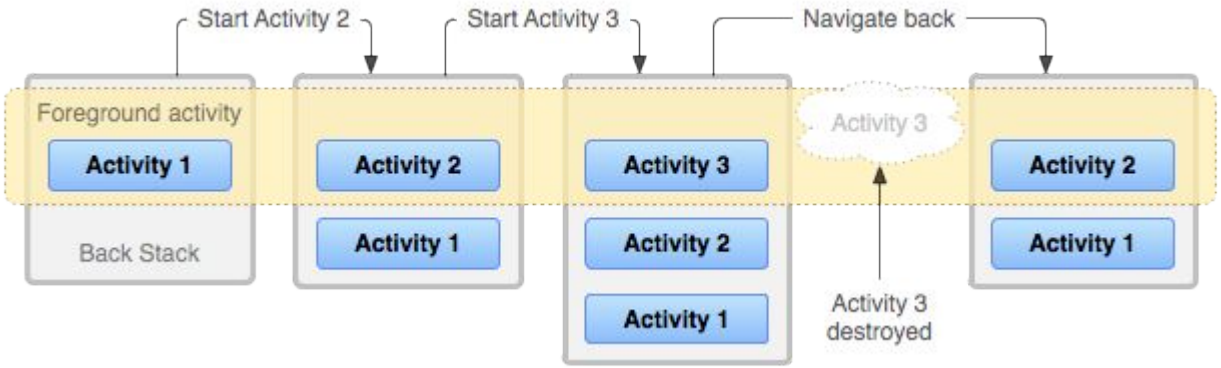
Context

- Application
  - Single instance
  - Extends Context
  - *Used for*: Instantiating storage (Room, Data Store, …), obtaining system services, …
  - Cannot be used for: UI related operations (like showing dialogs)
- Activity/Service
  - Multiple instances
  - Extends Context
  - Can be easily leaked
  - *Used for:* showing dialogs, showing toasts/snackbars, accessing resources, inflating layouts, creating Intents, …

Activity: Task stack & Lifecycle

Tasks and back stack

- Task is collection of activities, to perform certain job
- Activity in task can be from different application (send email)
- Activities arranged in a stack, in order in which there were opened
- Task has its own back stack
- Changing behaviour of task stack:
  - Manifest attributes (`taskAffinity`, `launchMode`, ...)
  - Intent flags



Intent Flags for task stack manipulation

FLAG_ACTIVITY_NEW_TASK

- Start activity in new task, or bring task with that activity

FLAG_ACTIVITY_CLEAR_TOP

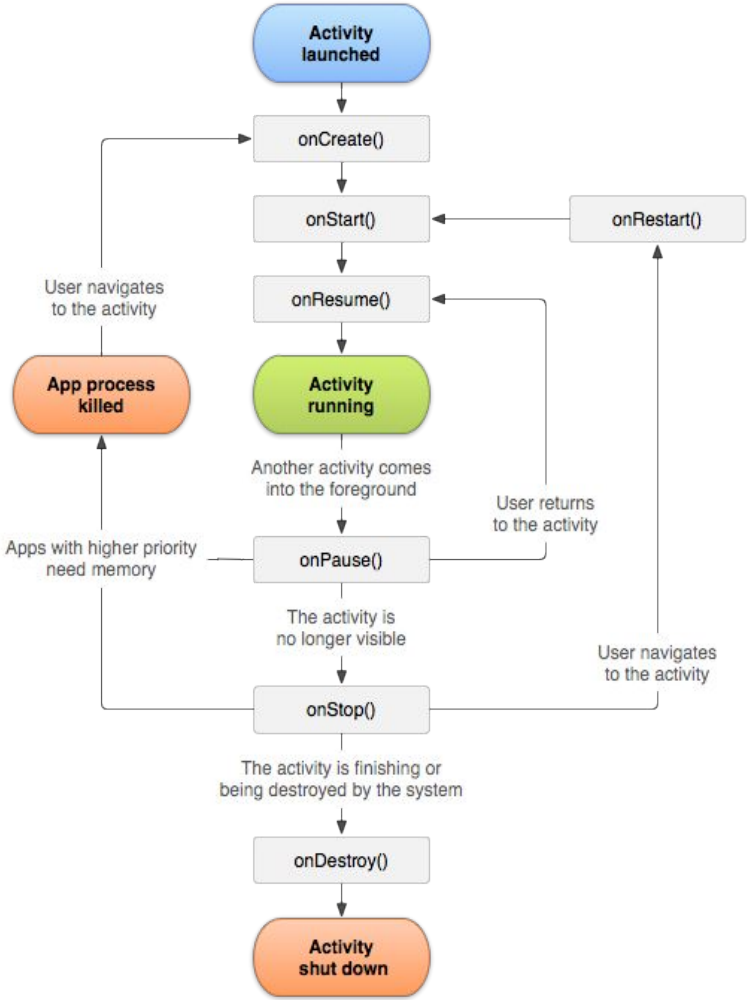
- If the activity is in stack, pick them and destroy all other activities on top

FLAG_ACTIVITY_SINGLE_TOP

- Do not start new instance of activity, if is already on top of stack

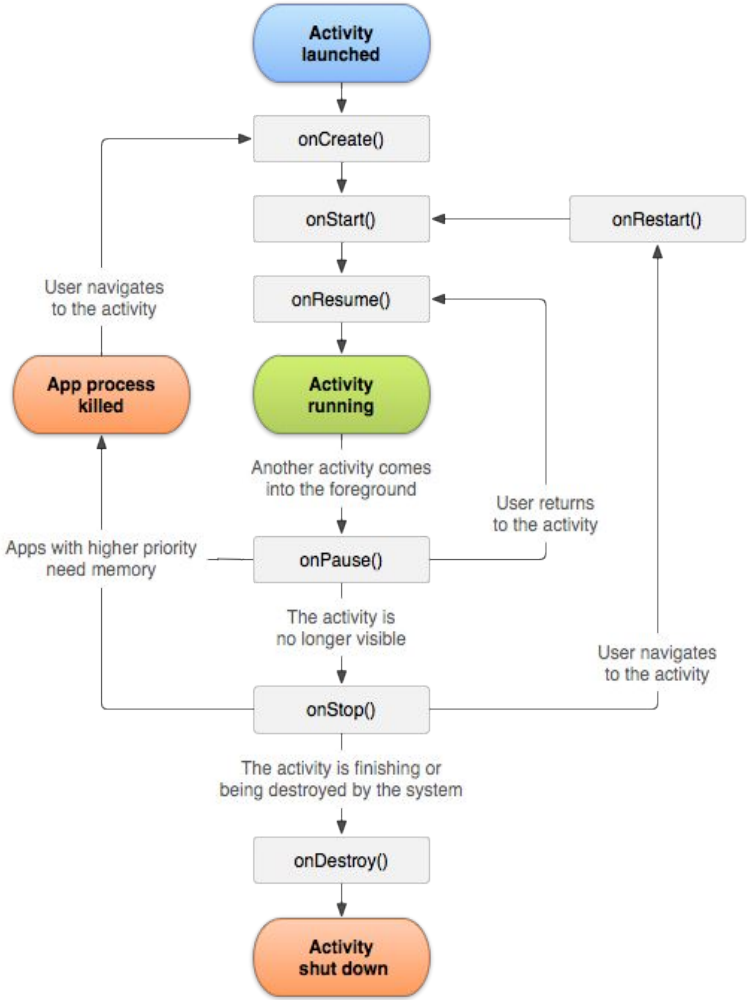
Activity: States

- Documentation
- **Activity changes state based on the user or OS actions:**
  - User navigates to activity
  - User switches to different app and returns
  - User presses back button
  - Screen is automatically locked
  - Phone starts ringing
  - ...
- **Lifecycle callbacks:**
  - Methods called by OS when state of activity changes
  - Allows programmer to react to these changes



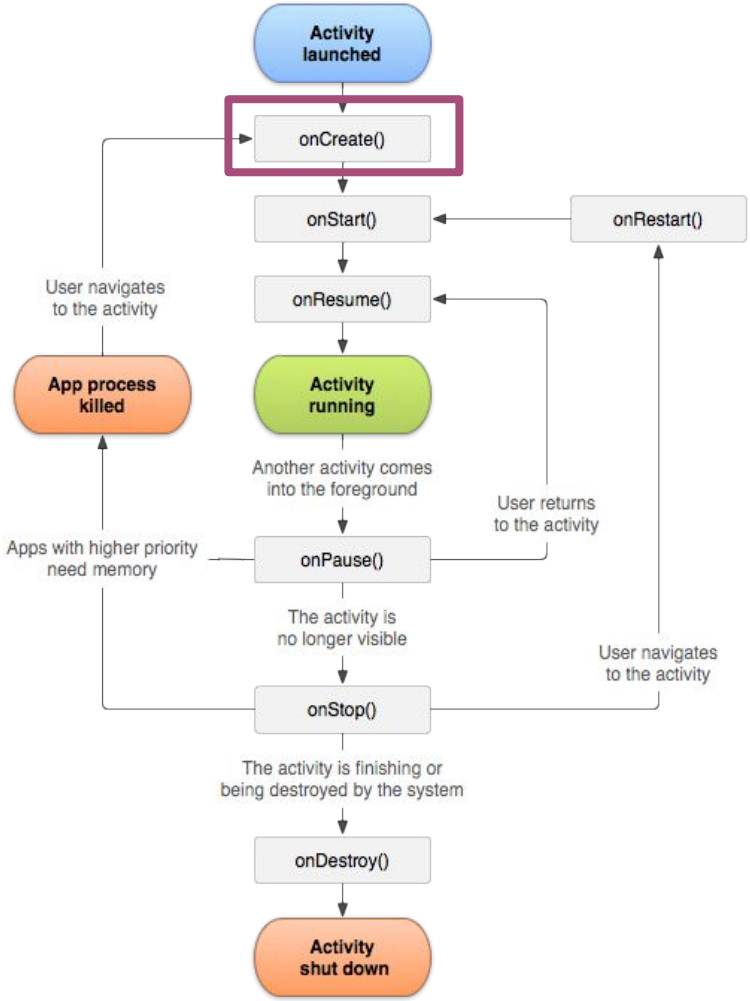
Activity: States

- **Created**: Activity is being created
- **Started**: Activity is about to be visible
- **Resumed**: Running, is visible, user can interact
- **Paused**: Partially visible, remains in memory
- **Stopped**:
  - Different activity is on top
  - Moved to background
  - Still alive, remains in memory
  - Hosting process can be killed
- **Destroyed**



Activity#onCreate(Bundle)

- Activity is being created
- One-time event, called only once per instance
- Create views
- Passed Bundle object contains activity previous state
- Read data from starting intent
- Always followed by #onStart()



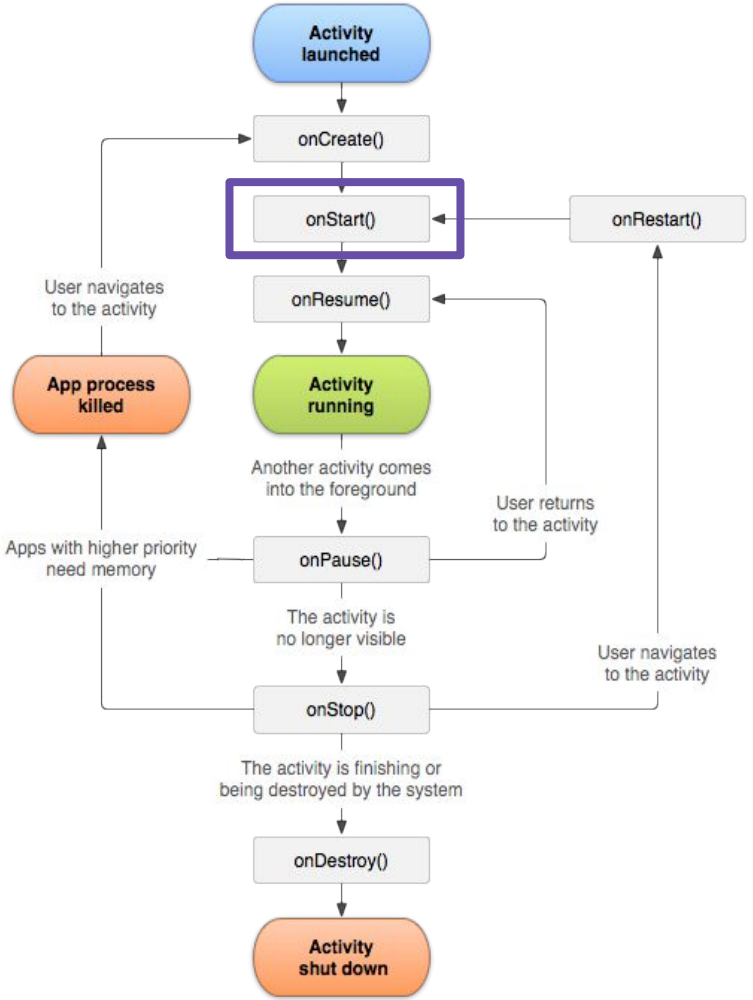
Activity#onCreate(Bundle)

SomeActivity#onCreate

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    previousState = savedInstanceState?.getString(STATE_KEY)
    setContentView(R.layout.main_activity)
}
```

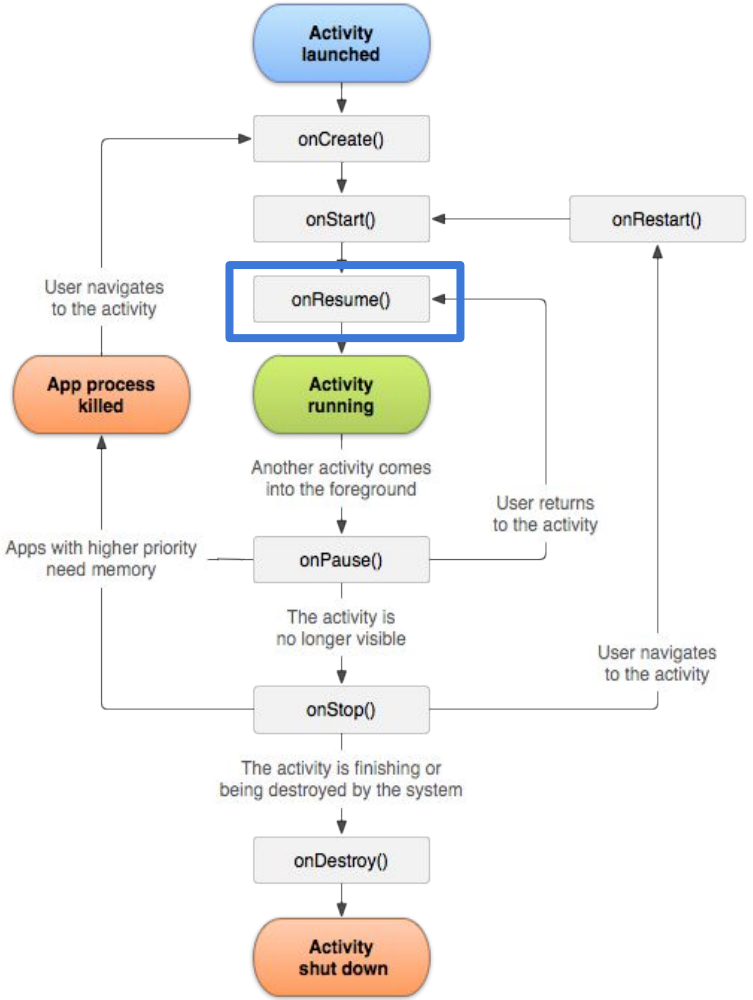
Activity#**onStart()**

- Called before the activity become visible to the user
- Can be called multiple times
- Followed by
  - **onResume**() if come to the foreground
  - **onStop**() if becomes hidden
- Activity is partially visible, register listeners for changing UI
- Register broadcast receivers



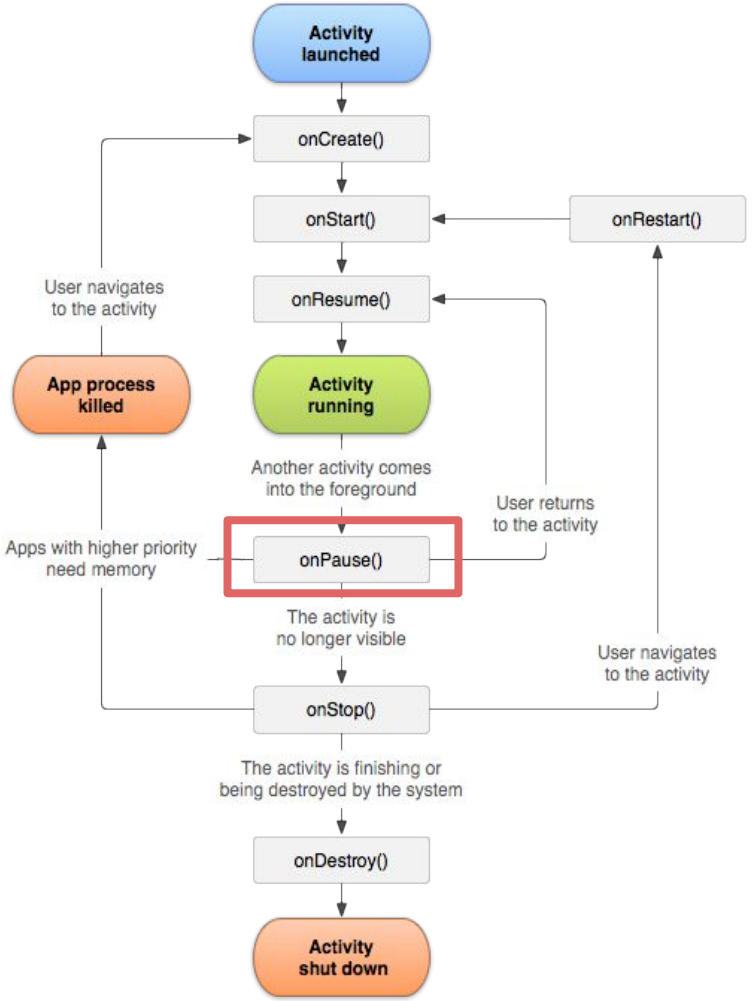
Activity#**onResume()**

- Called just before activity start interacts with user
- Activity is on top of activity stack
- Run stuff for user
- Always followed by **onPause**()



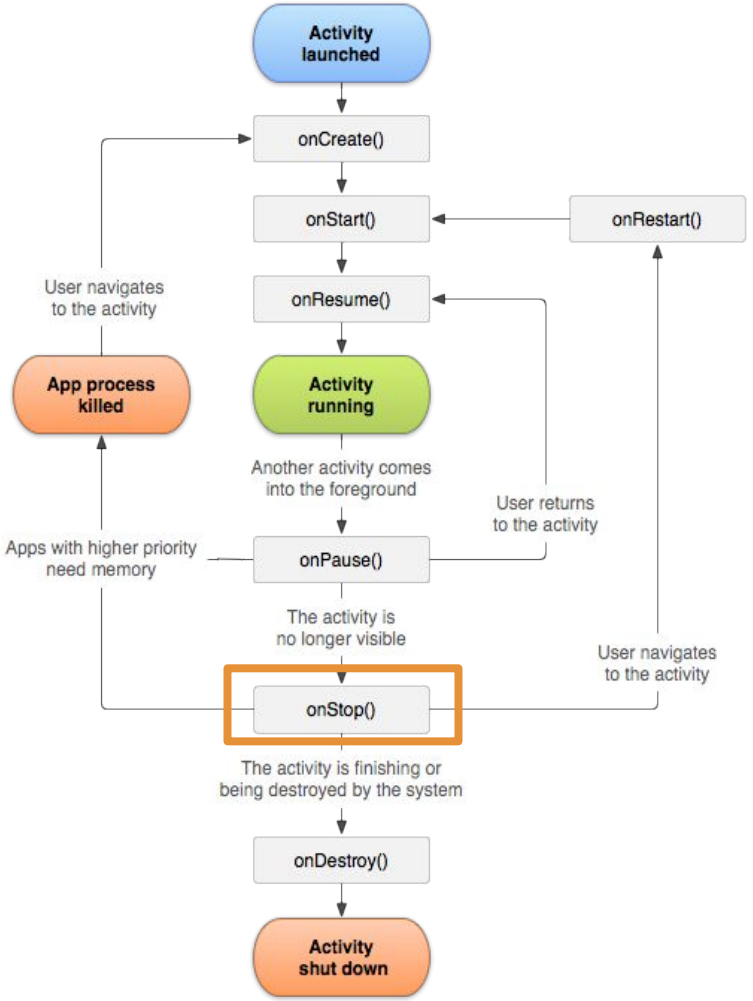
Activity#onPause()

- System is about to resume another activity
- Stop animations and CPU intensive stuff
- Should be very fast, because another activity onResume() waits until this finishes
- Followed by
  - **onResume**() if the activity returns back to the front
  - **onStop**() if became invisible to the user
- Activity can be killed by system
- Counterpart to onResume()



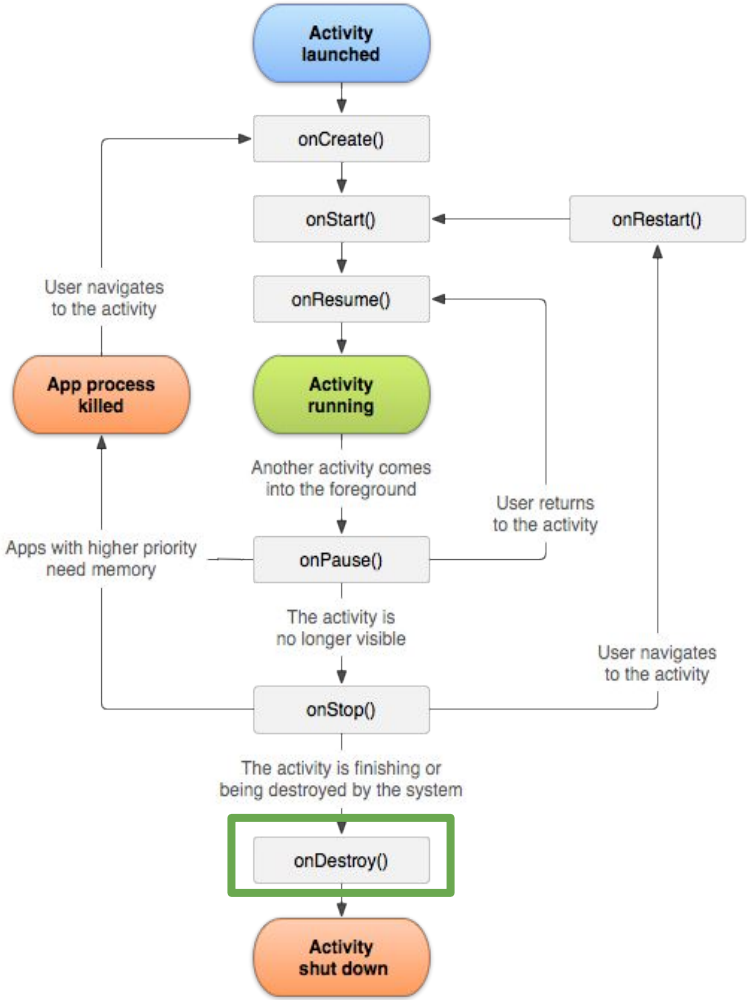
Activity#onStop()

- Called when it is no longer visible to the user
- It is being destroyed or another activity has been resumed and covering it.
- Finish stuff started in #onStart()
- Followed by
  - **onRestart**() - coming back to interact with user
  - **onDestroy**() - activity is going away
- Called when being minimized, navigate to another screen



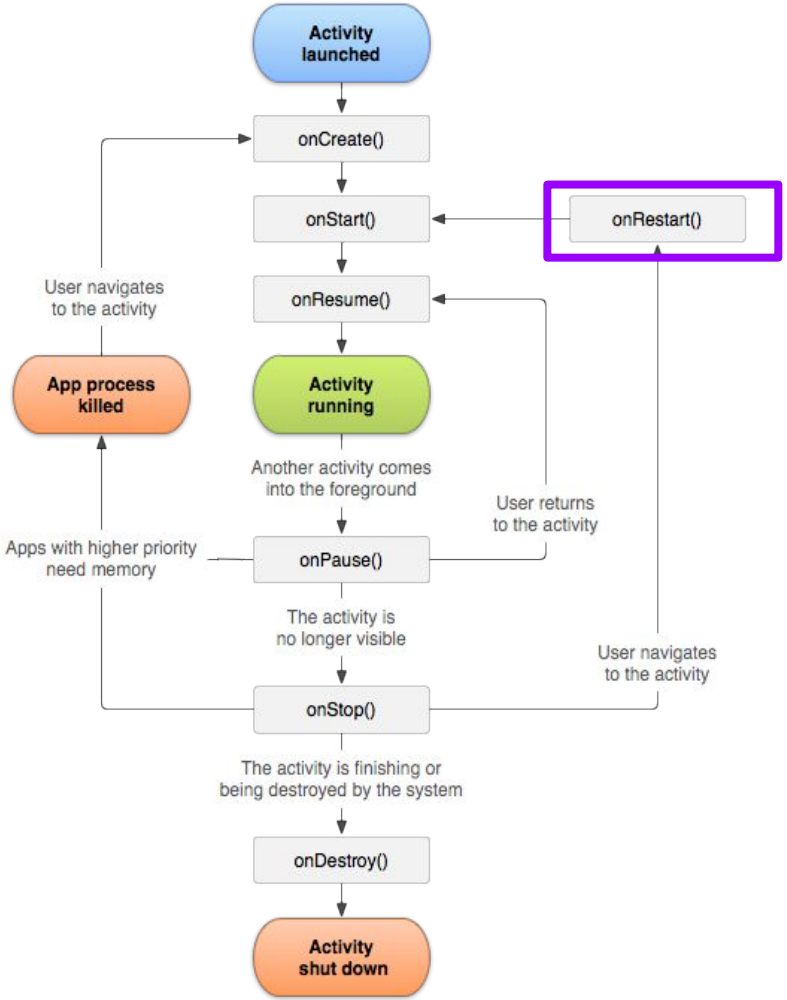
Activity#onDestroy()

- Called before activity is destroyed
- Activity is finished by #finish() method
- System needs more resources (RAM)



Activity#onRestart()

- Called after activity has been stopped, and before is started again



- Activity lifecycles demo

Fragments

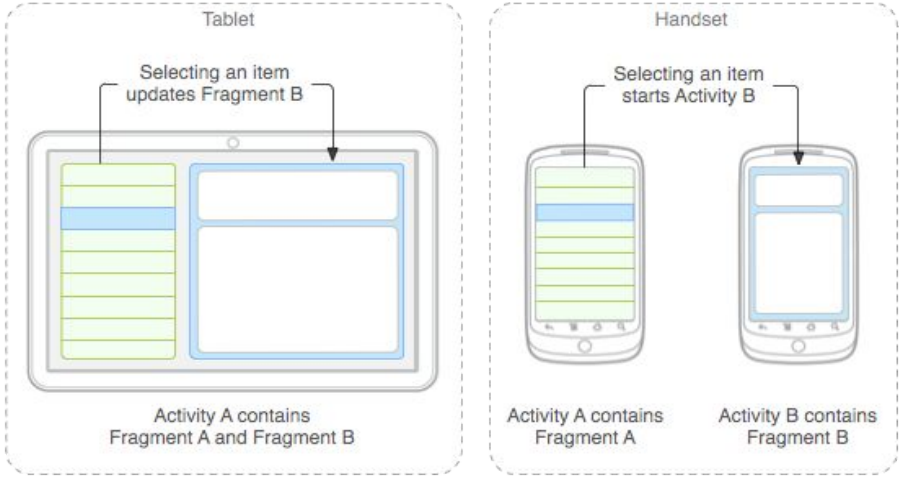
Fragment

- Simplify creating UI for phones and tablets
- Needs to be inside of an Activity
- **For most cases should not be used together with Jetpack Compose**

~~android.app.Fragment~~ - Added API 11, deprecated API 28

~~android.support.v4.app.Fragment~~ - replaced by Jetpack

androidx.fragment.app.Fragment - Jetpack



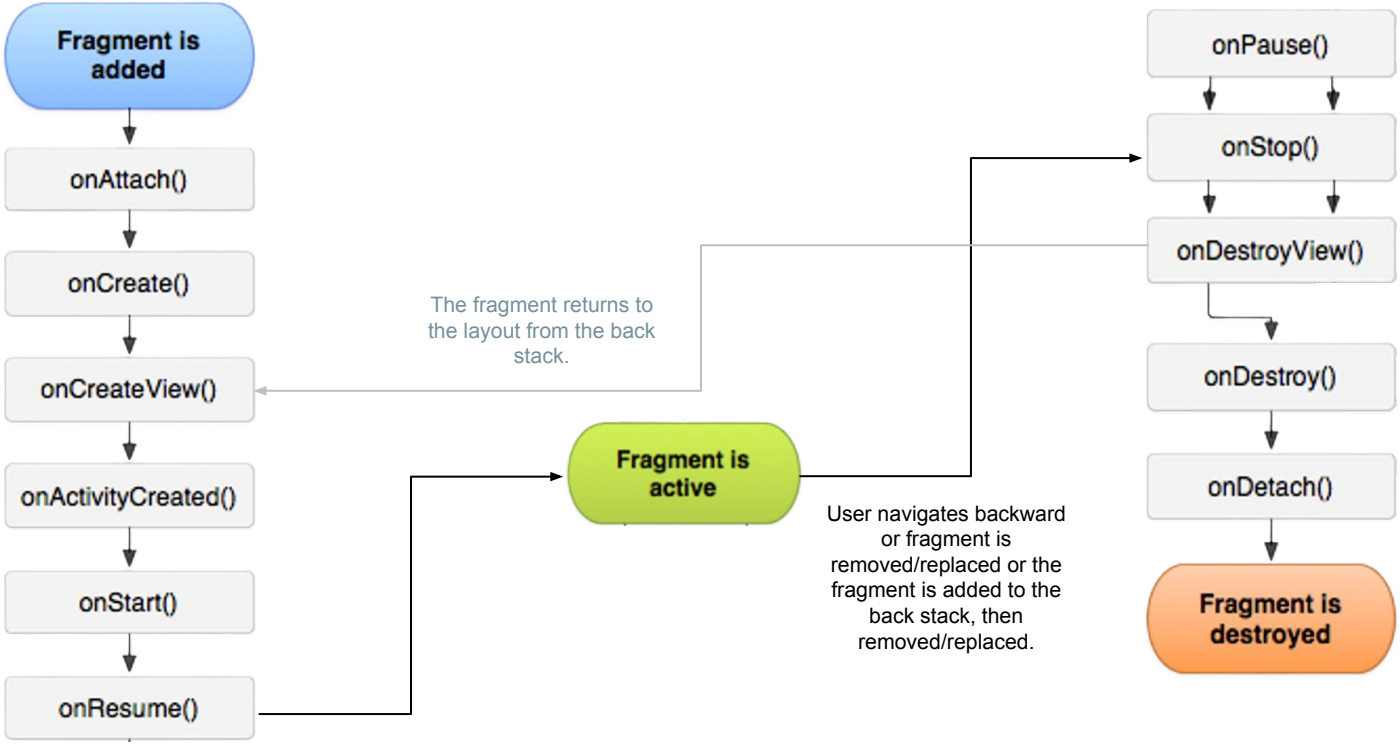
Fragment: States

- Is in same state as host activity
- **Resumed**
  - Fragment is visible in the running activity
- **Paused**
  - Another activity in foreground, but hosting activity is still visible
- **Stopped**
  - Fragment is not visible
  - Hosting activity is stopped or fragment is removed from the activity, but added to backstack.
  - Alive, can be killed by hosting activity

Fragment: Lifecycle

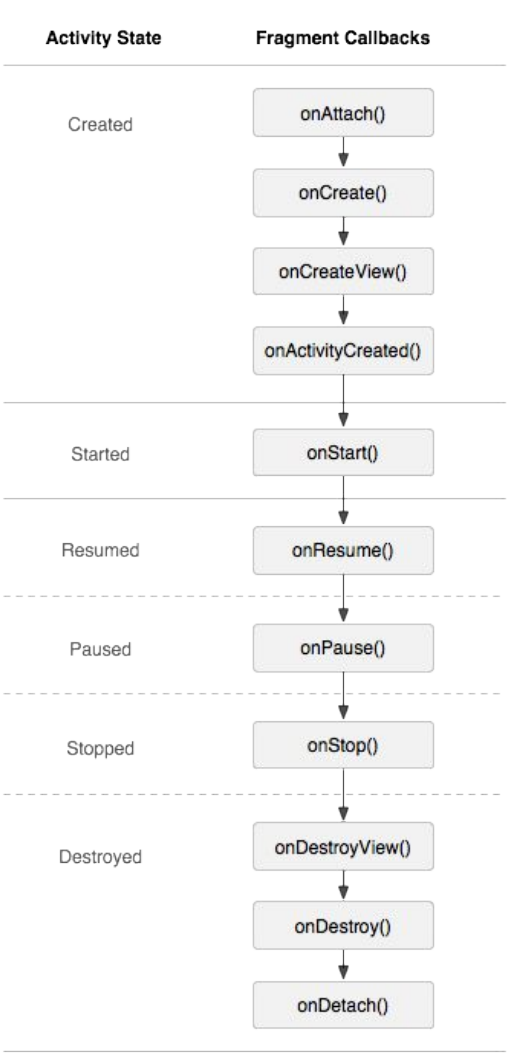
- Is in same state as host activity
- **Resumed**
  - Fragment is visible in the running activity
- **Paused**
  - Another activity in foreground, but hosting activity is still visible
- **Stopped**
  - Fragment is not visible
  - Hosting activity is stopped or fragment is removed from the activity, but added to backstack.
  - Alive, can be killed by hosting activity

Fragment: Lifecycle



Fragment: Inside Activity lifecycle

- Fragment's lifecycle is closely tied to its hosting Activity's lifecycle.
- When the Activity moves through its lifecycle states, the fragment transitions accordingly.
- Fragments start and resume along with the Activity, ensuring UI consistency. However, they manage their views separately and handle cleanup in onDestroyView()



Fragment: Add statically

activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragment_id"
        android:tag="some_string"
        android:name="packagename.class"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</FrameLayout>
```

Fragment: Add dynamically

Activity.kt

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    setContentView(R.layout.activity_user)

    supportFragmentManager.beginTransaction()
            .add(R.id.fragment_container, MyFragment.newInstance())
            .addToBackStack(null)
            .commit(
}
```

User Interface: Implementation

Approaches to writing UI

- **Old way:** XML
- **New way:** Jetpack Compose

XML: Layouts

- Definition of UI
- Used for Activity or Fragment
- Defined in XML or programmatically
- Folder res/layout
- *Options*: FrameLayout, LinearLayout, RelativeLayout , TableLayout, GridLayout, ConstraintLayout (Google IO 2016, Available as support library)
- **Not recommended to use in new projects (use Jetpack Compose)**

XML: Binding between layouts and java

- XML elements has id generated in R.java:
- R.id.txt_headline
- R.layout.activity_main

- Binding
  - Manual
  - View binding – preferred way
    - https://developer.android.com/topic/libraries/view-binding
  - Data binding
  - Kotlin synthetics (deprecated)

Layout - FrameLayout

- Places all items in top left corner
- Usage as placeholder for other view/fragment
- Fast

XML

```xml
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:src="@drawable/sample_image" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, FrameLayout!" />

</FrameLayout>
```

Layout - LinearLayout

- Places childs vertically or horizontally (orientation)
- Possible to use weight to size item in some ratio
- Usually leads to layout nesting

XML

```xml
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, LinearLayout!" />

    <ImageView
        android:layout_width="wrap_content"
        android:layout_height="200dp"
        android:src="@drawable/sample_image" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me" />

</LinearLayout>
```

Layout - Constraint layout

- “Extended relative layout”
- Constraint is connection or alignment to another view/parent/guideline
- Recommended today
- Documentation
- Available as dependency:

"androidx.constraintlayout:constraintlayout"

XML

```xml
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, ConstraintLayout!"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me"
        app:layout_constraintTop_toBottomOf="@id/textView"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Widgets – UI elements

- Extend View
- width and height needs to be set
  - Can be replaced by weight
  - match_parent: Fills the whole width/height of parent
  - wrap_content: Wraps around the content
  - Specific dimension

- Button
- TextView
- EditText
- ImageView
- CheckBox
- RadioButton
- WebView
- AdapterView
  - ListView
  - Spinner
- RecyclerView

Navigation

- *Navigation* = how user moves between "screens"
- *Options*:
  - Between activities: Intents
  - Between fragments: fragment transactions, navigation component

Up Next:
Better way to do UI

Thank you for attention

Feel free to leave feedback about this lesson: